

Website Automation Agent

An autonomous, AI-driven browser agent that fills web forms by itself — powered by Google Gemini (a multimodal LLM) in a perceive→think→act loop.

Python

Playwright

Google Gemini

Function Calling

Vision / Multimodal

Agentic Loop

1 What the project is

An AI agent that controls a real web browser by itself to fill out a form. It is a mini version of tools like "Browser Use" / computer-use agents. You give it a URL and a goal in plain English, and a **Generative-AI model (Gemini)** looks at the page, reasons about it, and drives Chromium — clicking, typing, scrolling — until the task is done. **Nothing is hardcoded:** the AI decides every action.

Target task: open `https://ui.shadcn.com/docs/forms/react-hook-form`, identify the form fields, and fill the Name and Description fields automatically.

2 The big idea: an "agent", not a script

A normal **script** is rigid ("click `#name`, type X") and breaks when the page changes. An **agent** follows a **perceive → think → act** loop, like a human:

```
PERCEIVE  take a screenshot + read the form fields      <---+
          |
THINK     Gemini reasons: "which field is Name? which is |
          Description? what should I type?"             |
          |
ACT       call a tool (click / type / scroll)  -----+
repeat until the AI calls "finish"
```

This loop is the heart of the project and the core GenAI idea — an **agentic loop** (a ReAct-style "Reason + Act" agent).

3 Architecture (files & roles)

```
main.py          entry point: loads config, logging, starts the agent
config.py        settings from .env (API key, model chain, URL, task)
agent/
|- browser_tools.py  the "hands & eyes": Playwright wrapper (the tools)
|- tool_schemas.py  tool descriptions, written FOR the AI to read
|- agent.py          the "brain": the Gemini reasoning loop
```

Think **brain + hands**: the model *decides* (`agent.py`), the code *executes* (`browser_tools.py`). This separation is a key agent-design principle.

4 Generative-AI logic implemented

These are the GenAI concepts to know cold for the viva.

1 LLM-powered agentic loop (Reason + Act)

The model is called repeatedly in a loop; each turn it reasons and picks one action, the result is fed back, and it reasons again. Ends when it calls the `finish` tool. `MAX_STEPS` is a safety cap.

2 Tool Use / Function Calling — the most important concept

Instead of plain text, the model outputs a structured **function call** such as `click_on_screen(x=640, y=398)`. Tools are *declared* to Gemini in `tool_schemas.py` (name + description + typed parameters). The model **chooses** which tool and which arguments — that is the intelligence. The 9 tools: `open_browser`, `navigate_to_url`, `take_screenshot`, `get_form_fields`, `click_on_screen`, `double_click`, `send_keys`, `scroll`, `finish`.

3 Multimodal AI / Vision

Gemini can read images. After each action we send a fresh **screenshot** as an image, so the model literally **sees** the page and reasons over it — a "computer-use"-style agent.

4 Prompt Engineering (system prompt)

The `SYSTEM_PROMPT` is the model's job description: it frames the task, the method (look→reason→act→verify), and hands decisions to the AI ("you — not any script — decide which element is which").

5 Grounding with structured data

Guessing pixel coordinates from an image is unreliable, so `get_form_fields` runs JavaScript to return each field's label, placeholder, type and exact centre `(x, y)`. The model still *decides* which field is which; we only feed it accurate raw facts.

6 Conversation-state management

The API is **stateless**, so each turn we resend the full history (messages, the model's prior tool calls, tool results, the latest screenshot). That is the agent's "memory" of what it has already done.

7 Multi-model fallback (reliability)

Each Gemini model has its own quota. On `429 RESOURCE_EXHAUSTED` (or a 500/503) the agent rotates to the next model in a chain (2.5-flash → 2.0-flash → 2.5-flash-lite → 2.0-flash-lite), "sticky" once it falls back.

8 Token-aware context management

Old screenshots are pruned from history so only the **latest** image is resent each turn — images are token-heavy, and this keeps the free-tier quota from draining. The model still sees the current page.

9 Robust, self-correcting behaviour

Tool errors (timeout, off-screen element) are returned to the model **as feedback**, so it adapts (e.g. scroll and retry) instead of crashing.

5 How one full run plays out

On the live page the field is labelled **"Bug Title"**, not literally "Name" — the AI works that out itself. A typical trajectory:

- `open_browser` → Chromium launches.
- `navigate_to_url(target)` → page loads; screenshot returned to model.
- `get_form_fields()` → a "Bug Title" input + a "Description" textarea, with coordinates.
- Model **reasons**: "Bug Title is the primary/name field; the textarea is the description."
- `click_on_screen(x, y)` on Bug Title → `send_keys("Login button broken on mobile")`.
- Description is below the fold → `scroll` down, then `get_form_fields()` again (coordinates change after scrolling — the model knows to re-check).
- `click_on_screen` + `send_keys` on Description.
- `get_form_fields()` again to **verify** the values.
- `finish("Filled the title and description fields.")` → loop ends.

The exact steps are the model's choice each run — that's the point. It is not a fixed script.

6 Design choices (likely questions)

- **Why Playwright?** First-class Python API, auto-waits for elements, reliable screenshots, easy Chromium management. (Recommended by the assignment.)
- **Why feed coordinates instead of pure vision?** Reliability — the model still decides; we give it accurate pixel targets so `click_on_screen(x, y)` lands. `device_scale_factor=1` makes screenshot pixels map 1:1 to clicks.
- **Why is this "intelligent"?** Element identification and decisions are done by AI reasoning over labels + types + the screenshot, not hardcoded `if` rules.
- **Why a manual loop, not auto function-calling?** So we can inject a fresh screenshot between turns (vision feedback) and log every decision.

7 The required tools → where they live

All in `agent/browser_tools.py`, exposed to the AI as function calls.

Required tool	What it does
<code>open_browser</code>	launches Chromium
<code>navigate_to_url</code>	goes to a URL
<code>take_screenshot</code>	captures the viewport (also fed to the AI as vision)
<code>click_on_screen(x, y)</code>	mouse click at coordinates
<code>double_click</code>	double-click (select existing text before overwriting)
<code>send_keys</code>	type text into the focused field
<code>scroll</code>	scroll to reveal hidden fields
<code>get_form_fields</code> (bonus)	intelligent element detection feeding the AI

8 30-second summary to open your viva

"It's an autonomous AI agent that fills web forms. It uses Google Gemini — a multimodal LLM — in an agentic perceive-think-act loop. The required browser tools (open, navigate, screenshot, click, type, scroll, double-click) are exposed to the model via **function calling**; Gemini looks at a **screenshot** of the page plus structured field data, **reasons** about which field is which, and **decides** what action to take each step — nothing is hardcoded. It feeds errors back to the model so it self-corrects, and it falls back across multiple Gemini models if one hits its quota."